

Brayan Smith Gil Álvarez

Ing. De sistemas

DESARROLLO DE TALLER DE LABORATORIO – TÉCNICAS DE ADMINISTRACIÓN DE MEMORIA

1. Contexto del problema:

Una empresa de desarrollo de software está implementando un sistema para gestionar la memoria de un servidor que ejecuta simultáneamente múltiples aplicaciones.

El servidor cuenta con una memoria principal limitada y debe alojar diversos procesos de diferentes tamaños. El equipo de ingeniería debe determinar cuál técnica de administración de memoria permite un mejor aprovechamiento de los recursos disponibles.

2. Objetivos:

- Comprender las principales técnicas de administración de memoria utilizadas por los sistemas operativos.
- Analizar las ventajas y desventajas de cada técnica.
- Aplicar conceptos de administración de memoria a un problema práctico.
- Justificar técnicamente la elección de una estrategia de administración de memoria.

3. Investigación Conceptual

Realice la investigación de los siguientes conceptos:

Intercambio (Swapping)

- ¿Qué es?

El intercambio o *swapping* es una técnica de administración de memoria utilizada por los sistemas operativos para optimizar el uso de la memoria RAM. Consiste en trasladar temporalmente procesos o programas desde la memoria principal hacia un espacio especial del disco llamado área de intercambio (*swap*) cuando la RAM disponible no es suficiente. Esto permite que el sistema pueda ejecutar más procesos de los que cabrían únicamente en la memoria principal.

- ¿Cómo funciona?

Cuando la memoria RAM se llena, el sistema operativo selecciona uno o varios procesos que no están siendo utilizados en ese momento y los mueve al área de intercambio en el disco, liberando espacio para otros procesos activos. Más adelante, cuando alguno de esos procesos sea necesario nuevamente, el sistema lo devuelve a la memoria RAM y reanuda su ejecución desde el punto en que fue suspendido. De esta manera, el sistema mantiene varios procesos disponibles, aunque la memoria física sea limitada.

- **Ventajas**
El intercambio (swapping) permite aprovechar mejor la memoria RAM, ya que posibilita la ejecución de más procesos de los que cabrían únicamente en la memoria principal. Además, mejora la multiprogramación y ayuda a mantener el sistema funcionando incluso cuando la memoria disponible es limitada.
- **Desventajas**
El principal inconveniente es que el acceso al disco es mucho más lento que el acceso a la memoria RAM, por lo que mover procesos entre ambos puede disminuir el rendimiento del sistema. Si el intercambio ocurre con demasiada frecuencia, puede producirse un fenómeno llamado *thrashing*, donde el sistema dedica más tiempo a mover procesos que a ejecutarlos, causando una notable lentitud.

Asignación Contigua

- **¿Qué es?**
La **asignación contigua** es una técnica de administración de memoria en la que cada proceso se almacena en un único bloque continuo de la memoria RAM. Esto significa que toda la memoria necesaria para un proceso debe estar disponible en una sola sección consecutiva para que pueda cargarse y ejecutarse.
- **¿Cómo se asigna la memoria?**
Cuando un proceso solicita memoria, el sistema operativo busca un bloque libre lo suficientemente grande para almacenarlo de forma continua. Si encuentra un espacio adecuado, asigna ese bloque completo al proceso. Existen diferentes estrategias para hacerlo, como **First Fit** (primer ajuste), **Best Fit** (mejor ajuste) y **Worst Fit** (peor ajuste).
- **¿Qué es la fragmentación interna?**
La **fragmentación interna** ocurre cuando a un proceso se le asigna un bloque de memoria mayor al que realmente necesita. Como resultado, una parte del espacio asignado queda sin utilizar dentro del bloque, desperdiciando memoria.
- **¿Qué es la fragmentación externa?**
La **fragmentación externa** ocurre cuando la memoria libre queda dividida en varios bloques pequeños dispersos. Aunque la suma de estos espacios libres pueda ser suficiente para un nuevo proceso, este no puede cargarse porque no existe un bloque contiguo lo bastante grande para alojarlo. Esto dificulta el aprovechamiento eficiente de la memoria.

Paginación

- **¿Qué es una página?**
Una página es una unidad de tamaño fijo en la que se divide la memoria lógica de un proceso. Cada proceso se fragmenta en varias páginas para facilitar su almacenamiento y administración en la memoria principal.
- **¿Qué es un marco de página?**
Un **marco de página** es un bloque de memoria física de tamaño fijo donde se almacena una página. La memoria RAM se divide en marcos de página y cada página de un proceso puede ubicarse en cualquier marco disponible, sin necesidad de que estén juntos físicamente.
- **¿Cómo se realiza la traducción de direcciones?**
La **traducción de direcciones** se realiza mediante una **tabla de páginas** administrada por el sistema operativo. Cuando un proceso genera una dirección lógica, esta se divide en un número de página y un desplazamiento. El sistema consulta la tabla de páginas para encontrar el marco de memoria física donde está almacenada esa página y combina el número de marco con el desplazamiento para obtener la dirección física exacta en la RAM. Esto permite que los procesos utilicen la memoria de forma eficiente sin requerir almacenamiento contiguo.

Segmentación

- **¿Qué es un segmento?**
Un **segmento** es una unidad lógica de memoria de tamaño variable que representa una parte específica de un programa. A diferencia de la paginación, la segmentación divide un programa según su estructura lógica, permitiendo separar diferentes tipos de información y facilitando su organización y protección.
- **¿Qué tipos de segmentos puede tener un programa?**
Un programa puede estar dividido en varios segmentos, entre los más comunes se encuentran:
 - **Segmento de código:** contiene las instrucciones o el código ejecutable del programa.
 - **Segmento de datos:** almacena variables globales y datos estáticos utilizados por el programa.
 - **Segmento de pila (stack):** guarda variables locales, parámetros de funciones y direcciones de retorno de llamadas.
 - **Segmento de montón (heap):** se utiliza para la asignación dinámica de memoria durante la ejecución del programa.Esta división permite que cada parte del programa sea gestionada de forma independiente por el sistema operativo.

Segmentación + Paginación

- **¿Cómo combina ambas técnicas?**

La **segmentación con paginación** combina las ventajas de la segmentación y la paginación. Primero, el programa se divide en **segmentos lógicos** (como código, datos, pila y montón), y luego cada segmento se divide en **páginas** de tamaño fijo. Estas páginas se almacenan en marcos de página de la memoria física. De esta manera, se mantiene la organización lógica de la segmentación y la flexibilidad de la paginación para administrar la memoria.

- **¿Qué ventajas ofrece frente a las técnicas anteriores?**

La segmentación con paginación permite una mejor organización de los programas al mantener separados sus distintos componentes lógicos, facilita la protección y el control de acceso a cada segmento, y reduce el problema de la **fragmentación externa** gracias al uso de páginas de tamaño fijo. Además, aprovecha de forma más eficiente la memoria física y ofrece mayor flexibilidad para ejecutar procesos grandes y complejos en comparación con la asignación contigua, la paginación o la segmentación por separado.

4. Problema Algorítmico

Contexto

Una computadora dispone de 1024 MB de memoria RAM. Durante una jornada de trabajo llegan los siguientes procesos:

Proceso	Tamaño (MB)
P1	120
P2	250
P3	80
P4	300
P5	150
P6	200

El sistema debe determinar si cada proceso puede cargarse en memoria y cómo administrarlo utilizando alguna de las técnicas estudiadas.

Parte A. Diseño del Algoritmo

Diseñe un algoritmo que permita:

- Registrar los procesos.
- Calcular la memoria total utilizada.
- Calcular la memoria libre.
- Determinar si un nuevo proceso puede cargarse o no.
- Mostrar un reporte final de ocupación de memoria.

R// El algoritmo en Pseudocódigo fue desarrollado en PSeInt y se encuentra adjunto en el correo de entrega del taller 2.

```
=====
PARTE A - ADMINISTRACION DE MEMORIA
=====

Memoria Total: 1024 MB
Procesos cargados:
P1 = 120 MB
P2 = 250 MB
P3 = 80 MB
P4 = 300 MB
P5 = 150 MB
Memoria utilizada: 900 MB
Memoria disponible: 124 MB
P6 NO puede cargarse.
```

Parte B. Simulación de Asignación Contigua

Modifique el algoritmo para representar la memoria como una lista de bloques.

Ejemplo:

| P1 | P2 | P3 | Espacio libre |

El algoritmo debe:

- Mostrar el orden de ubicación de los procesos.
- Indicar la memoria libre restante.
- Determinar si existe espacio suficiente para un nuevo proceso.

R// El algoritmo en Pseudocódigo fue desarrollado en PSeInt y se encuentra adjunto en el correo de entrega del taller 2.

```
=====
PARTE B - ASIGNACION CONTIGUA
=====
```

```
| P1 | P2 | P3 | P4 | P5 | Espacio Libre |
Memoria utilizada: 900 MB
Memoria libre: 124 MB
P6 NO puede cargarse.
```

Parte C. Simulación de Paginación

Suponga páginas de 50 MB.

El algoritmo debe:

- Calcular cuántas páginas necesita cada proceso.
- Mostrar el resultado.

Ejemplo:

P1 = 120 MB

Páginas necesarias:

$120 / 50 = 3$ paginas

Salida:

P1 -> 3 paginas

P2 -> 5 paginas

P3 -> 2 paginas

R// El algoritmo en Pseudocódigo fue desarrollado en PSeInt y se encuentra adjunto en el correo de entrega del taller 2.

```
=====
PARTE C - PAGINACION
=====
```

```
Tamano de pagina: 50 MB
P1 -> 3 paginas
P2 -> 5 paginas
P3 -> 2 paginas
P4 -> 6 paginas
P5 -> 3 paginas
P6 -> 4 paginas
```

Parte D. Simulación de Segmentación

R// El algoritmo en Pseudocódigo fue desarrollado en PSeInt y se encuentra adjunto en el correo de entrega del taller 2.

```
=====
PARTE D - SEGMENTACION
=====
Proceso: P1
Codigo: 60 MB
Datos: 40 MB
Pila: 20 MB
Total: 120 MB
```

Parte E. Selección de Técnica

Análisis de resultados:

Al realizar las simulaciones se observó que la memoria total disponible es de 1024 MB. Los procesos P1, P2, P3, P4 y P5 ocupan 900 MB de memoria, quedando únicamente 124 MB libres. Debido a esto, el proceso P6, que requiere 200 MB, no puede cargarse mediante asignación contigua.

En la simulación de paginación se comprobó que los procesos pueden dividirse en páginas de tamaño fijo, permitiendo una mejor organización de la memoria y evitando problemas de fragmentación externa.

En la simulación de segmentación se observó que un proceso puede dividirse en segmentos lógicos como Código, Datos y Pila, facilitando la organización y administración de los recursos.

Selección de técnica:

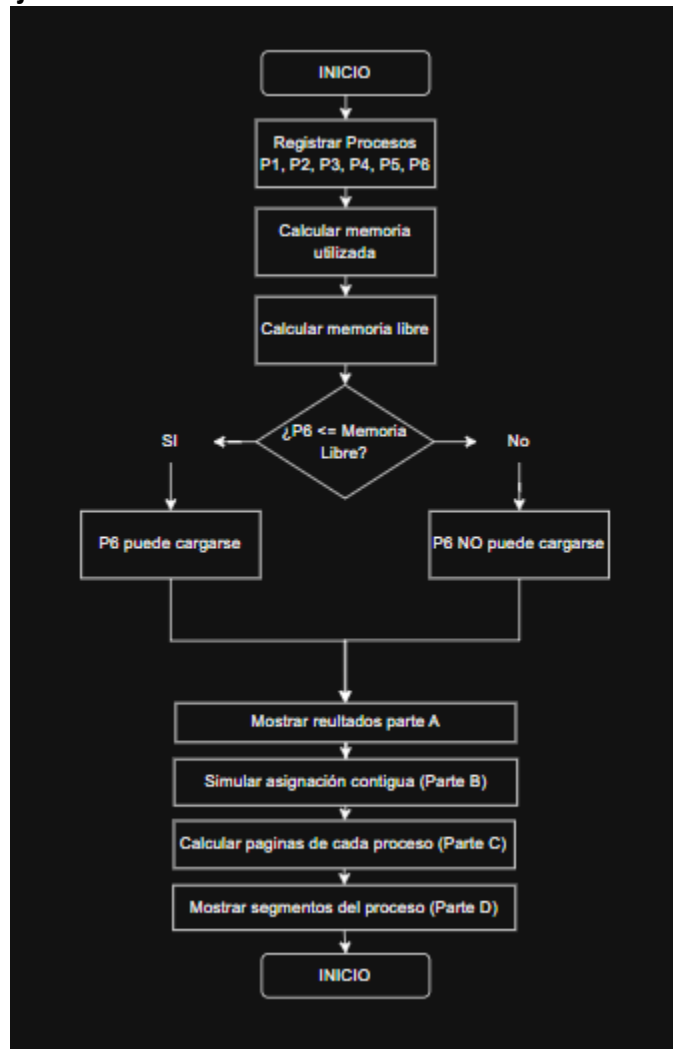
Le mejor técnica en este caso es **Segmentación + Paginación**

Justificación:

La técnica seleccionada es Segmentación + Paginación, ya que combina las ventajas de ambas estrategias. La segmentación permite dividir los procesos en partes lógicas como código, datos y pila, facilitando la organización y protección de la memoria. Por su parte, la paginación divide los segmentos en bloques de tamaño fijo, reduciendo la fragmentación externa y mejorando el aprovechamiento de la memoria disponible.

En comparación con la asignación contigua, esta técnica utiliza la memoria de manera más eficiente. Además, evita las limitaciones del intercambio (swapping), que puede disminuir el rendimiento debido al acceso constante al disco. Por estas razones, Segmentación + Paginación resulta la alternativa más adecuada para un sistema que ejecuta múltiples procesos simultáneamente.

5. Diagrama de flujo:



6. Reflexión final:

Si fuera el arquitecto de un sistema operativo moderno, implementaría la técnica de Segmentación + Paginación para la administración de memoria. Esta técnica permite organizar los procesos de manera lógica mediante segmentos y, al mismo tiempo, aprovechar eficientemente la memoria física gracias a la paginación.

Además, reduce los problemas de fragmentación, mejora la protección de los datos y facilita la ejecución simultánea de múltiples procesos. Estas características la convierten en una solución eficiente, flexible y escalable para los sistemas operativos actuales.